

Advanced Software for Data science : Julia, R, Python and Excel

Python

Colombe Becquart
colombe.becquart@tse-fr.eu



Virtual Environment

Why use a Python Environment?

- **Dependency Management:**

- Isolate project dependencies
- Avoid conflicts between libraries or Python versions in different projects

Example: For a new project, you want to use the latest version of Matplotlib to take advantage of its new features. However, an older project depends on a previous version of Matplotlib, where some functions may have been deprecated. By creating separate environments, you can ensure that each project uses its own compatible version of Matplotlib without causing conflicts between the two.

- **Reproducibility:**

- Ensures that the code runs consistently across different machines and setups

Types of Python Environments

- **Global Environment:**
 - Default one
 - Not recommended for project-specific setups
- **Local Environments:**
 - Let you install packages specific to your workspace without affecting other environments
 - Virtual Environments
 - Conda Environments

Virtual Environments

- **Venv** is a module that comes preinstalled with Python 3.3 and later, **no additional installation required**.
- **Lightweight:** quick and simple setup with minimal configuration
- Can be created using `python -m venv myenv`
- [doc](#)

Conda Environments

- **Conda:**
 - Open-source package and environment management system that supports multiple programming languages, including Python
 - Part of the Anaconda distribution
 - Can also be used independently through Miniconda ([Should I use Anaconda Distribution or Miniconda?](#))
- **Conda Environments:**
 - **Cross-language support:** Handles packages from various languages like Python, R, and C++
 - **Comprehensive package manager:** Manages dependencies and packages, including non-Python libraries
 - **Robust:** Suitable for complex projects requiring multiple languages and extensive dependencies
 - Can create environments using `conda create --name myenv python=3.x`
 - [doc](#)

Sharing Environments

- **requirements.txt**

- **Format:** a plain text file listing all the packages and their versions. Each line contains a package name, optionally followed by a version number.
- Save the current environment dependencies: `pip freeze > requirements.txt`
- Install dependencies from a requirements.txt file: `pip install -r requirements.txt`
- Or create an environment from an requirements.txt file: `conda create --name my_env --file requirements.txt`

- **environment.yml**

- **Format:** a YAML file that defines a full environment, including its name, the Python version, and the packages to install and their versions.
- Export your active environment to a new file: `conda env export > environment.yml`
- Create an environment from an environment.yml file: `conda env create -f environment.yml`

Best Practices for Managing Python Environments

- Always use virtual environments for your projects
- Keep your `requirements.txt` or `environment.yml` up-to-date
- Activate the correct environment before running your code
- Clean up unused environments to save disk space
- Environment names:
 - Descriptive names
 - Avoid spaces
 - Python version: if you're using multiple Python versions across different projects